DINKY ASWANI
T11-06

Aim: To perform version control operations using Git.

LO 02: Introduction to Git, GitHub, Basic operations: clone, commit, push, pull, Branching

strategies and merging, Git workflows: Feature branching, Git Flow, Handling merge

conflicts, GitHub Actions basic

Git Commands — Version Control Experiment

Setup & Configuration

1. git --version — Checks whether Git is installed on the system and displays the installed version. This command is used before starting the experiment to ensure that Git has been installed correctly.

2. git config --global user.name "Your Name" — Sets the username that Git attaches to every commit. This username identifies the author of all future commits made from the system.

3. git config --global user.email "your@email.com" — Sets the email address associated with every commit. The configured email is stored with each commit and helps identify the contributor.

Creating the Repository

4. mkdir git_experiment — Creates a new folder named git_experiment. This folder serves as the project directory where all Git operations will be performed.

5. cd git_experiment — Changes the current working directory to the newly created project folder so that Git commands are executed inside the repository.

6. git init — Initializes a new Git repository by creating a hidden .git folder. This folder stores all repository information, including commits, branches, and configuration files.

7. git status — Displays the current state of the repository by showing tracked files, untracked files, modified files, and staged files. It helps determine the next Git operation.

Creating & Committing Files

8. echo 'print("Hello World - Version 1")' > app.py — Creates a Python file named app.py containing the first version of the program. If the file already exists, its contents are replaced

9. git add app.py — Stages the app.py file by moving it to the staging area. Only staged files are included in the next commit.

10. git commit -m "Version 1" — Creates the first commit by saving the staged changes permanently in the repository. The commit message describes the changes made.

11. git log --oneline — Displays the commit history in a compact one-line format, showing the commit ID and commit message for each saved version.

Editing & Comparing Versions

12. echo 'print("Hello World - Version 2")' > app.py — Updates the contents of app.py to create a second version of the program, allowing Git to detect changes.

13. git diff — Compares the modified file with the most recent committed version and displays all added, removed, or modified lines.

14. git add . — Stages all modified and newly created files in the current directory instead of adding each file individually.

15. git commit -m "Version 2: Updated message" — Saves the updated project as a new commit, creating another version in the repository history.

Branching

16. git branch — Lists all available branches in the repository and indicates the currently active branch with an asterisk (*).

17. git branch feature-branch — Creates a new branch named feature-branch without switching to it. This branch is used to develop new features independently.

18. git checkout feature-branch — Switches the working directory from the current branch to feature-branch so that further changes are made only in that branch.

19. echo 'print("Hello World - Version 3 on feature branch")' > app.py — Modifies app.py while working on feature-branch, creating a separate version of the file.

20. git commit -am "Version 3: Edited on feature branch" — Stages all tracked modified files and commits them in a single command, saving the changes made on the feature branch.

21. git checkout master — Switches back to the master branch, allowing further work to continue on the main project branch.

22. git merge feature-branch — Merges all committed changes from feature-branch into the currently active branch. If no conflicts exist, Git performs the merge automatically.

23. git checkout -b test-branch — Creates a new branch named test-branch and immediately switches to it using a single command.

Undoing Changes / Rollback

24. git restore app.py — Discards all uncommitted changes made to app.py and restores the file to the state of the last committed version.

25. git add app.py — Stages the modified app.py file before demonstrating how to remove it from the staging area.

26. git restore --staged app.py — Removes app.py from the staging area while keeping the changes in the working directory unchanged.

27. git reset --soft HEAD~1 — Removes the latest commit while preserving all changes in the staging area so they can be recommitted if required.

28. git reset --hard HEAD~1 — Removes the latest commit along with all associated changes from both the repository and the working directory. This operation permanently discards the changes.

29. git revert <commit-id> — Creates a new commit that reverses the changes introduced by an earlier commit without deleting the existing commit history, making it the safest rollback method.

Viewing Final State

30. git log --oneline — Displays the final commit history after completing all Git operations, allowing verification of the complete sequence of project versions.

1 – Check Git Installation

```
C:\Users\DINKY ASWANI>git --version
git version 2.49.0.windows.1
```

2 – Configure Username

```
C:\Users\DINKY ASWANI>git config --global user.name "Dinky Aswani"
```

3 – Configure Email

```
C:\Users\DINKY ASWANI>git config --global user.email "dinkyaswani1812@gmail.
com"
```

4 – Create Project Folder

```
C:\Users\DINKY ASWANI>mkdir git_experiment
```

5 – Open the Folder

```
C:\Users\DINKY ASWANI>cd git_experiment
```

6 – Initialize Git Repository

```
C:\Users\DINKY ASWANI\git_experiment>git init
Initialized empty Git repository in C:/Users/DINKY ASWANI/git_experiment/.git/
```

7 – Check Repository Status

```
C:\Users\DINKY ASWANI\git_experiment>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

8 – Create First File

```
C:\Users\DINKY ASWANI\git_experiment>echo print("Hello World - Version 1") > app.py

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.py

nothing added to commit but untracked files present (use "git add" to track)
```

9 – Stage the File

```
C:\Users\DINKY ASWANI\git_experiment>git add app.py
```

10 – Create First Commit

```
C:\Users\DINKY ASWANI\git_experiment>git commit -m "Version 1"
[master (root-commit) 4e1f799] Version 1
 1 file changed, 1 insertion(+)
 create mode 100644 app.py
```

11 – View Commit History

```
C:\Users\DINKY ASWANI\git_experiment>git log --oneline
4e1f799 (HEAD -> master) Version 1
```

12 – Modify the File

```
C:\Users\DINKY ASWANI\git_experiment>echo print("Hello World - Version 2") > app.py
```

13 – Compare Changes

```
C:\Users\DINKY ASWANI\git_experiment>git diff
diff --git a/app.py b/app.py
index d0912ba..437ac5a 100644
--- a/app.py
+++ b/app.py
@@ -1 +1 @@
-print("Hello World - Version 1")
+print("Hello World - Version 2")

C:\Users\DINKY ASWANI\git_experiment>dir
 Volume in drive C is Windows
 Volume Serial Number is A4AC-9906

 Directory of C:\Users\DINKY ASWANI\git_experiment

15-07-2026  02:05    <DIR>          .
15-07-2026  02:07    <DIR>          ..
15-07-2026  02:07                35 app.py
               1 File(s)             35 bytes
               2 Dir(s)  22,389,518,336 bytes free

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py
```

14 – Stage All Changes

```
C:\Users\DINKY ASWANI\git_experiment>git add .

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

15 – Commit Updated Version

```
C:\Users\DINKY ASWANI\git_experiment>git commit -m "Version 2: Updated message"
[master 62747c8] Version 2: Updated message
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\DINKY ASWANI\git_experiment>git log --oneline
62747c8 (HEAD -> master) Version 2: Updated message
4e1f799 Version 1

C:\Users\DINKY ASWANI\git_experiment>git add .

C:\Users\DINKY ASWANI\git_experiment>git commit -m "Version 2: Updated message"
On branch master
nothing to commit, working tree clean

C:\Users\DINKY ASWANI\git_experiment>git log --oneline
62747c8 (HEAD -> master) Version 2: Updated message
4e1f799 Version 1
```

16 – Create a New Branch

```
C:\Users\DINKY ASWANI\git_experiment>git branch feature-branch

C:\Users\DINKY ASWANI\git_experiment>git branch
  feature-branch
* master
```

17 – Switch to Feature Branch

```
C:\Users\DINKY ASWANI\git_experiment>git checkout feature-branch
Switched to branch 'feature-branch'

C:\Users\DINKY ASWANI\git_experiment>git branch
* feature-branch
  master
```

18 – Modify File on Feature Branch

```
C:\Users\DINKY ASWANI\git_experiment>echo print("Hello World - Version 3 on feature branch") > app.py
```

19 – Commit Changes on Feature Branch

```
C:\Users\DINKY ASWANI\git_experiment>git commit -am "Version 3: Edited on feature branch"
[feature-branch c3c4e4e] Version 3: Edited on feature branch
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\DINKY ASWANI\git_experiment>git log --oneline
c3c4e4e (HEAD -> feature-branch) Version 3: Edited on feature branch
62747c8 (master) Version 2: Updated message
4e1f799 Version 1
```

20 – Return to Master Branch

```
C:\Users\DINKY ASWANI\git_experiment>git checkout master
Switched to branch 'master'

C:\Users\DINKY ASWANI\git_experiment>git merge feature-branch
Updating 62747c8..c3c4e4e
Fast-forward
 app.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\DINKY ASWANI\git_experiment>git branch
  feature-branch
* master

C:\Users\DINKY ASWANI\git_experiment>git log --oneline
c3c4e4e (HEAD -> master, feature-branch) Version 3: Edited on feature branch
62747c8 Version 2: Updated message
4e1f799 Version 1

C:\Users\DINKY ASWANI\git_experiment>git checkout -b test-branch
Switched to a new branch 'test-branch'

C:\Users\DINKY ASWANI\git_experiment>git checkout master
Switched to branch 'master'
```

21 – Merge Feature Branch

```
C:\Users\DINKY ASWANI\git_experiment>git merge feature-branch
Already up to date.

C:\Users\DINKY ASWANI\git_experiment>git branch
  feature-branch
* master
  test-branch
```

22 – Create and Switch to Test Branch

```
C:\Users\DINKY ASWANI\git_experiment>git checkout -b test-branch
Switched to a new branch 'test-branch'

C:\Users\DINKY ASWANI\git_experiment>git checkout master
Switched to branch 'master'

C:\Users\DINKY ASWANI\git_experiment>git merge feature-branch
Already up to date.

C:\Users\DINKY ASWANI\git_experiment>git branch
  feature-branch
* master
  test-branch

C:\Users\DINKY ASWANI\git_experiment>git log --oneline
c3c4e4e (HEAD -> master, test-branch, feature-branch) Version 3: Edited on feature branch
62747c8 Version 2: Updated message
4e1f799 Version 1
```

23 – Restore File

```
C:\Users\DINKY ASWANI\git_experiment>git restore app.py

C:\Users\DINKY ASWANI\git_experiment>type app.py
print("Hello World - Version 3 on feature branch")

C:\Users\DINKY ASWANI\git_experiment>echo another temp change > app.py

C:\Users\DINKY ASWANI\git_experiment>type app.py
another temp change

C:\Users\DINKY ASWANI\git_experiment>git add app.py

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

24 – Unstage a File

```
C:\Users\DINKY ASWANI\git_experiment>git restore --staged app.py

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

25 – Undo Commits

```
C:\Users\DINKY ASWANI\git_experiment>git reset --soft HEAD~1

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py


C:\Users\DINKY ASWANI\git_experiment>git commit -m "Temp commit for hard reset demo"
[master 12e1733] Temp commit for hard reset demo
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\DINKY ASWANI\git_experiment>git reset --hard HEAD~1
HEAD is now at c3c4e4e Version 3: Edited on feature branch

C:\Users\DINKY ASWANI\git_experiment>git status
On branch master
nothing to commit, working tree clean
```

View Final History

```
C:\Users\DINKY ASWANI\git_experiment>git log --oneline
c3c4e4e (HEAD -> master, test-branch, feature-branch) Version 3: Edited on feature branch
62747c8 Version 2: Updated message
4e1f799 Version 1
```

Conclusion

The experiment demonstrated the basic operations of Git, including repository creation, file tracking, committing changes, branching, merging, and rollback. It showed how Git maintains project history and allows developers to manage different versions of a project efficiently. This practical exercise helped in understanding the importance of version control in software development.